

Handoff — 500px-style Pulse Stats & Scoring

Date: 2026-06-06

Branch: feat/pulse-stats-500px (off admin)

For: dev ที่จะรับงานต่อ / apply migration / deploy

เอกสารนี้สรุป **ทุกขั้นตอน** ที่ทำในรอบนี้: ระบบคะแนน Pulse, การเก็บคะแนน, และ stats panel แบบ 500px พร้อม **สิ่งที่ dev ต้องทำ** ก่อนใช้งานจริง

1. TL;DR

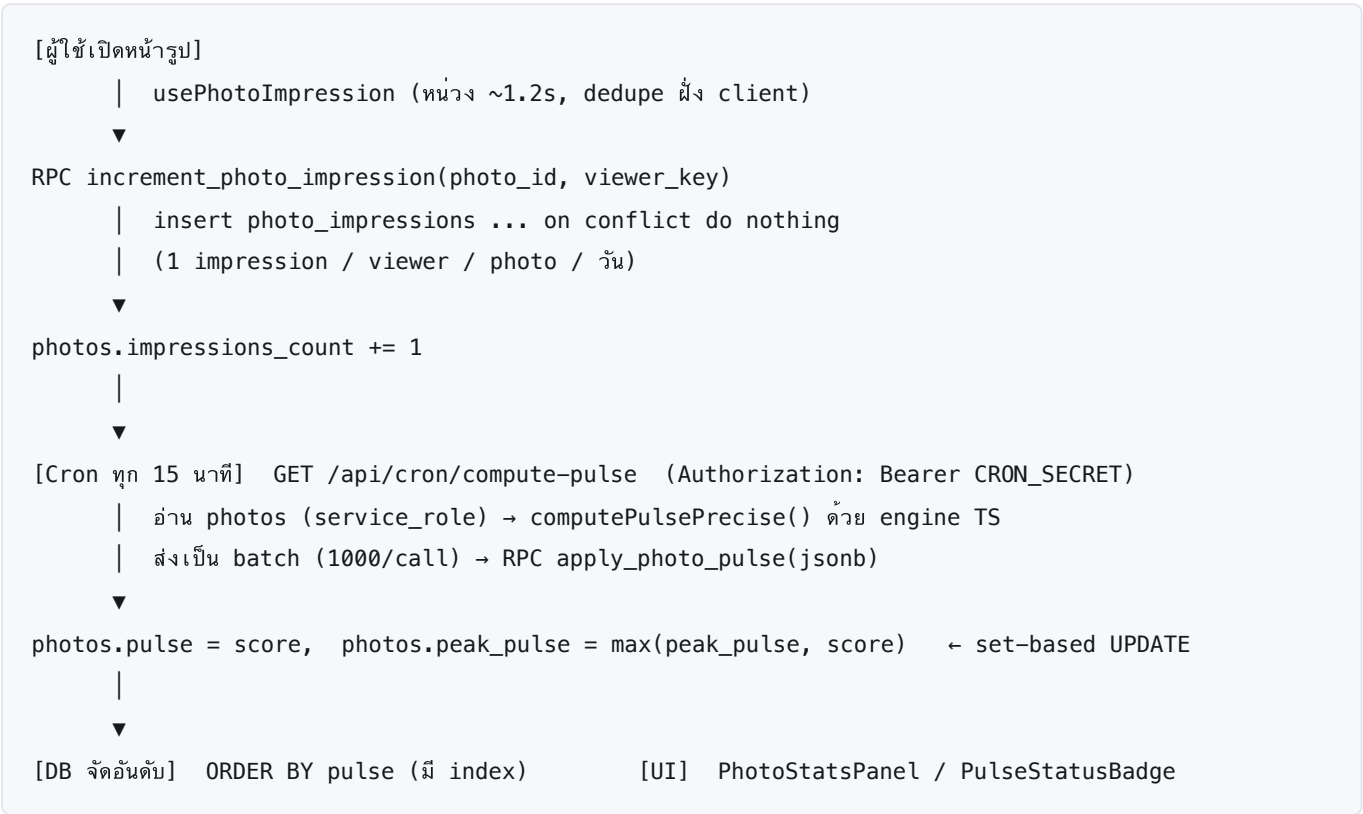
เป้าหมาย: ทำให้หน้ารูปโซเชียลมีเดียแบบ 500px — Likes / Impressions / Highest Pulse / Status (Popular) และทำให้ DB จัดอันดับด้วย pulse ได้

ปัญหาเดิมที่แก้:

- 1. คะแนน Pulse คำนวณใน JS ฟังก์ชัน client เท่านั้น → ไม่เคยถูกเก็บลง DB → ORDER BY pulse ไม่ได้
- 2. มี 2 สูตรไม่ตรงกัน: engine TS (pulse-engine.ts , ฉลาด) vs DB materialized view (0003 , naive) → คะแนนโชว์ ≠ คะแนนจัดอันดับ
- 3. impressions_count มีคอลัมน์แต่ ไม่มีอะไรไปนับ → exposure score = 0 เสมอ

วิธีแก้ (สถาปัตยกรรม): **engine TS = source of truth เดียว** → cron คำนวณแล้ว persist ลงคอลัมน์ → DB จัดอันดับจากคอลัมน์นั้น

2. Data flow



****Highest Pulse = peak_pulse **** — เพราะ pulse มี decay (ขึ้นแล้วตก) ค่าที่ภูมิใจคือจุดพีค จึงเก็บ max ตลอดอายุรูป

3. สิ่งที่เราสร้าง (ไฟล์ทั้งหมด)

Phase 1 — Impression tracking

ไฟล์	หน้าที่
supabase/migrations/0014_pulse_persistence_and_impressions.sql	ตาราง photo_impressions (dedupe by photo+viewer+day) + RPC increment_photo_impression (SECURITY DEFINER, grant anon/authenticated)
src/hooks/usePhotoImpression.ts	client hook — ยิง RPC หลังอยู่หน้า คสข ~1.2s, viewer_key เก็บใน localStorage, กันยิงซ้ำด้วย sessionStorage

Phase 2 — Persist pulse + peak

ไฟล์	หน้าที่
0014_...sql	เพิ่มคอลัมน์ photos.pulse + photos.peak_pulse (numeric 5,1) + index pulse desc / peak_pulse desc
supabase/migrations/0015_apply_photo_pulse.sql	RPC apply_photo_pulse(jsonb) — set-based UPDATE (batch), peak = greatest(...), grant service_role เท่านั้น
src/app/api/cron/compute-pulse/route.ts	cron route — auth ด้วย CRON_SECRET, ใช้ service_role client, คำนวณด้วย engine, เขียน batch 1000/call
vercel.json	ตั้ง Vercel Cron */15 * * * * → /api/cron/compute-pulse
src/lib/pulse-engine.ts	เพิ่ม computePulsePrecise() (ทศนิยม 1 ตำแหน่ง "99.4") โดย computePulse() เดิมไม่เปลี่ยน + pulseStatus() + PULSE_STATUS_THRESHOLDS

Phase 3 — UI

ไฟล์	หน้าที่
<code>src/components/photo/PhotoStatsPanel.tsx</code>	panel 1000px (Likes / Impressions / Highest Pulse / Status)
<code>src/components/photo/PulseStatusBadge.tsx</code>	badge สถานะ monochrome (Rising/Popular/Editors' Choice), undiscovered = ไม่ใช้
<code>src/app/photo/[id]/PhotoDetailClient.tsx</code>	ต่อ panel (mobile = main column <code>lg:hidden</code> , desktop = sidebar <code>hidden lg:block</code>) + เรียก hook + map <code>impressions/peakPulse/pickType</code>
<code>src/components/photo/PhotoCard.tsx</code>	ใส่ <code>PulseStatusBadge</code> ได้ชื่อข้างภาพ → ครอบคลุม grid
<code>src/lib/types.ts</code>	เพิ่ม optional field <code>impressions?</code> / <code>peakPulse?</code> / <code>pickType?</code> ใน <code>Photo</code>
<code>src/messages/{en,th}.json</code>	namespace <code>PhotoStats</code> (title/likes/impressions/highest_pulse)

4. ⚠ สิ่งที่ dev ต้องทำก่อนใช้งานจริง (REQUIRED)

4.1 Apply migrations

รัน SQL บน Supabase ตามลำดับ:

```
supabase/migrations/0014_pulse_persistence_and_impressions.sql
supabase/migrations/0015_apply_photo_pulse.sql
```

(ผ่าน Supabase SQL editor หรือ `supabase db push` ตาม workflow ของทีม)

4.2 ตั้ง env (ทั้ง local `.env.local` และ Vercel Project Settings)

```
SUPABASE_SERVICE_ROLE_KEY=<Supabase → Settings → API → service_role>
CRON_SECRET=<สุ่มสตริงอะไรก็ได้>
```

- `SUPABASE_SERVICE_ROLE_KEY` = key ลับฝั่ง server (อย่า prefix `NEXT_PUBLIC_`, อย่า commit)
- `CRON_SECRET` = Vercel จะแนบเป็น `Authorization: Bearer <CRON_SECRET>` ให้อัตโนมัติเมื่อ cron ยิง

4.3 Deploy → Vercel cron จะรันเองทุก 15 นาที

ทดสอบ local ด้วยมือ:

```
curl -H "Authorization: Bearer <CRON_SECRET>" \
  http://localhost:3000/api/cron/compute-pulse
# คาดหวัง: {"ok":true,"scored":N,"updated":N}
# ไม่มี header → 401 (ถูกต้อง)
```

4.4 (ควรรำ) เลิกใช้สูตร velocity เดิมใน 0003

supabase/migrations/0003_materialized_views.sql ยังคำนวณ pulse_score ด้วยสูตรเก่า (likes/hours) และ 0004 ใช้ค่านั้นจัดอันดับ/cashback

→ เปลี่ยนให้ photo_scores / ranking อ่านจาก photos.pulse (ที่ engine เขียน) แทน เพื่อให้เหลือ source เดียว

5. Status tiers (ปรับได้)

pulseStatus(pulse, pickType) ใน pulse-engine.ts :

```
pickType != none          → Editors' Choice
pulse >= POPULAR (35)     → Popular
pulse >= RISING  (32)     → Rising
else                      → Undiscovered (ไม่โชว์ badge)
```

🔴 **สำคัญ:** ตอนนี้ PULSE_STATUS_THRESHOLDS = { RISING: 32, POPULAR: 35 } เป็นค่า **demo-tuned** เพราะ seed data มี pulse แค่ 26–36
พอมีข้อมูลจริง (likes + impressions จริง ทำให้ช่วงคะแนนกว้างขึ้น) **ต้องปรับขึ้นเป็น ~55 / 80** ไม่งั้น "Popular" จะเฟื่องทางที่ดีกว่า: เปลี่ยนเป็น **percentile-based** (top 10% = Popular) จะไม่ต้องจูนมือ

6. Open items / TODO

- **Apply migration 0014 + 0015** (ดู §4.1)
- ตั้ง env SUPABASE_SERVICE_ROLE_KEY + CRON_SECRET ใน Vercel (§4.2)
- **Retune status thresholds** หรือเปลี่ยนเป็น percentile (§5)
- เลิกใช้ velocity formula ใน 0003 , ให้ ranking อ่านจาก photos.pulse (§4.4)
- (option) status badge ใน **mobile masonry tile** (MobileExplore / MobileHome feed) — ตอนนี้ใส่แค่ PhotoCard (desktop grid)
- **Pre-existing type errors** ใน src/app/auth/callback/route.ts (nextUrl does not exist on Request) — ตัดมาก่อนหน้า ไม่เกี่ยวงานนี้ แต่ทำ next build fail ได้ ควรแก้ (ใช้ NextRequest แทน Request)
- **Voyageur → Traveller leftovers:** ยังมีคำศัพท์พิมพ์ใหญ่ VOYAGEUR ตกค้าง (messages exclusive , SideMenu.voyageurs , explore subtitle/aria-label) — ดู branch feat/rename-traveller

7. สถานะ branch (รอบงานนี้ทั้งหมด)

Branch	เนื้อหา	สถานะ
feat/pulse-stats-500px	งานในเอกสารนี้ (Pulse stats Phase 1-3 + batch/mobile/badge)	push แล้ว, ยังไม่ merge
feat/footer-unify	footer ใหม minimal "RANKING - SEASON 01" ทุกหน้า	merge เข้า admin แล้ว
feat/about-rewrite	เขียนหน้า About ใหม่ (Gography Photo Ranking)	merge เข้า admin แล้ว
feat/rename-traveller	rename Voyageur → Traveller ทั้ง UI + "การจัดอันดับ"	merge เข้า admin แล้ว (เหลือ leftover §6)

production = branch main (Vercel) · admin = integration branch

8. อ้างอิง

- Spec อัลกอริทึม: docs/specs/pulse-algorithm.md
- Engine: src/lib/pulse-engine.ts (พารามิเตอร์ทั้งหมดอยู่ใน PULSE_PARAMS)
- สูตร engine (ย่อ): pulse = clamp((engagement_log + exposure + metadata + pick) × decay, 19, 100)
- DB schema: supabase/migrations/0001_init_schema.sql (photos, votes), 0003 (views — กำลังจะ retire)